

Data Structures & Algorithms

Highlighted Study Summary – 60 Key Items

Chapters 1–10 Complete Coverage

GREEN 33 highlights **YELLOW** 6 highlights **RED** 4 highlights + 8 stamps (Ch.2) = 51 highlighted + 20 additional

Part 1: Highlighted items from PDFs (Chapters 1–10, including Ch.2 stamp marks)

Part 2: 20 Additional essential definitions

Prepared: June 2026

Chapter 1: Introduction to Python & Data Structures

Yellow Highlights — Foundational Concepts

YELLOW Data: A collection of facts and figures; a set of values in a particular format referring to a single set of item values. Can be stored in different forms such as text, numbers, and images.

YELLOW Data Structure (Linear vs Non-linear): A **linear** data structure has elements that combine to form a specific order (Arrays, Queues, Stacks, Linked Lists). Non-linear structures represent data with a hierarchical relationship (Trees, Graphs).

YELLOW Algorithm: A finite set of logic or instructions for solving a computational problem; data structures and algorithms are two core elements of every large and complex software project.

YELLOW if...else / elif Statement: Controls conditional execution of statements; format is a series of if and elif statements followed by a final else statement.

YELLOW Higher-Order Functions: Functions that take other functions as arguments or return functions as results. Python 3 built-in examples: filter() and map().

YELLOW Base Case (Recursion): To stop a recursive function from turning into an infinite loop, at least one argument **must** test for a terminating condition to end the recursion.

Green Highlights — Python Essentials

GREEN Python Variables (Dynamic Typing): In Python, it is **not required** to first declare the datatype for variables. Variables point to an object that can change type at any time during execution.

GREEN Function: A block of code which **only** runs when it is called. A function can receive data as parameters and return data as a result. Defined using the def keyword in Python.

GREEN Recursion: Occurs when a function makes one or more calls to itself during execution. Difference from iteration: recursion uses the call stack; iteration uses a loop.

GREEN Inheritance (OOP): Allows a new class to inherit functionality from other classes. Inheritance enables code reuse and modification of existing behavior without rewriting.

Exam Tip: Python uses duck typing — the type is determined at runtime, **not** compile time. This is called dynamic typing.

Exam Tip: A recursive function without a base case will cause a RecursionError (stack overflow).

Exam Trap: Higher-order functions (map, filter) are **not** for sorting — use sorted() with key= for sorting. Do **not** confuse them.

Exam Trap: **Linear** data structures are **NOT** the same as **linear** time $O(n)$. 'Linear' in DS means sequential ordering of elements.

Chapter 2: Python Data Types & Structures

Red Stamp Marks — Method Tables (Pages 10, 11, 15, 16, 18, 19, 21, 25)

RED Sequence Types — Common Methods (pp. 10–11): Key built-in functions for sequences (string, list, tuple, range): `len(s)` returns number of elements; `min(s)/max(s)` return smallest/largest; `sum(s)` returns sum of elements; `all(s)` returns True if all elements are True; `any(s)` checks whether any item in `s` is True.

RED Sequence Operations (p. 11): `s+r` concatenates two sequences of the same type; `s*n` makes `n` copies; `s[i]` indexing returns the `i`-th element; `s[i:j:stride]` slicing returns elements between `i` and `j`; `x in s` returns True if `x` is found; `x not in s` returns True if `x` is **not** in the sequence.

RED Dictionary Methods (p. 15): `len(d)` total items; `d.clear()` removes all items; `d.copy()` shallow copy; `d.get(k,v)` returns `d[k]` if found, else `v`; `d.items()` all key:value pairs; `d.keys()` all keys; `d.pop(k)` removes and returns `d[k]`; `d.update(b)` adds all objects from `b`; `d.values()` all values.

RED Dictionary — Key Rule (p. 16): Dictionary keys **must** be unique and immutable. Values can be any type. `d.setdefault(k,v)` returns `d[k]` if found; if **not** found, sets `d[k]=v` and returns `v`.

RED Set Methods — Immutable Operations (p. 18): `a.difference(t)`: elements in `a` but **not** in `t`; `a.intersection(t)`: elements in both; `a.isdisjoint(t)`: True if no element is common; `a.issubset(t)`: True if all `a` elements are in `t`; `a.issuperset(t)`: True if all `t` elements are in `a`; `a.union(t)`: elements in either `a` or `t`.

RED Mutable Set Methods (p. 19): `s.add(item)` adds item (nothing happens if already present); `s.clear()` removes all; `s.discard(item)` removes item; `s.remove(item)` deletes item; `s.pop()` returns an arbitrary item; `s.update(t)` appends all items from iterable `t`.

RED frozenset — Immutable Set (p. 21): Use `frozenset` instead of `set` when you want to add a set inside another set, or use it as a key in a dictionary. `s.add(frozenset(s2))` works; `s.add(s2)` raises `TypeError`.

RED Array vs List — Memory (p. 25): The `array` module stores typed data more efficiently than a Python list. `array.array('i', range(10**6))` uses ~91% less memory than `list(range(10**6))`. Use arrays when memory efficiency is **critical** for large numeric datasets.

Operation	Description
<code>s[i]</code>	Indexing — returns the <code>i</code> -th element
<code>s[i:j:stride]</code>	Slicing — returns elements between index <code>i</code> and <code>j</code>
<code>s+r</code>	Concatenation — combines two sequences of the same type
<code>x in s</code>	Membership test — True if <code>x</code> found in <code>s</code>
<code>len(s)</code>	Length — returns number of elements
<code>d.get(k,v)</code>	Safe dict access — returns <code>v</code> (not <code>KeyError</code>) if key missing
<code>a.intersection(t)</code>	Set intersection — elements in both <code>a</code> and <code>t</code>
<code>a.union(t)</code>	Set union — elements in either <code>a</code> or <code>t</code>

Exam Tip: `d.get(key, default)` is safer than `d[key]` — it **never** raises `KeyError`. **Always** use `get()` when the key might **not** exist.

Exam Tip: `frozenset` is hashable and can be used as a dictionary key or inside another set. Regular `set` **cannot**.

Exam Trap: `s.remove(item)` raises `KeyError` if item **not** found. Use `s.discard(item)` instead — it **never** raises an error.

Exam Trap: `array.array` is **NOT** the same as a Python list. It **only** stores elements of a single type but uses far less memory.

Chapter 3: Algorithm Analysis & Big-O Notation

Green Highlights

GREEN Divide and Conquer Paradigm: Involves: (1) breaking a problem into smaller simpler sub-problems, (2) solving those sub-problems independently, (3) combining their results.

GREEN Big-O Notation: The letter O stands for order, in recognition that growth rates are defined as the order of a function. Describes the upper bound of time/space complexity.

GREEN Common Complexity Classes (Growth Rates): From fastest to slowest: $O(1)$ **constant**, $O(\log n)$ **logarithmic**, $O(n)$ **linear**, $O(n \log n)$ linearithmic, $O(n^2)$ **quadratic**, $O(2^n)$ exponential.

Notation	Name	Example
$O(1)$	Constant	Array index lookup
$O(\log n)$	Logarithmic	Binary search
$O(n)$	Linear	Linear search
$O(n \log n)$	Linearithmic	Merge sort, Quick sort (avg)
$O(n^2)$	Quadratic	Bubble sort, Selection sort
$O(2^n)$	Exponential	Recursive Fibonacci

Exam Tip: Big-O **always** describes the WORST-case or upper bound. Big- Ω (Omega) describes the best case. Big- Θ (Theta) describes the exact bound.

Exam Trap: $O(2n)$ simplifies to $O(n)$ — constants are **always** dropped in Big-O notation. **Never** keep **constant** coefficients.

Chapter 4: Linked List Operations

Green Highlights

GREEN **Singly Linked List — Time Complexity of Operations:** The worst-case running time (time complexity) of a lookup/access by index is $O(n)$. Insert/delete at the beginning (with pointer) is $O(1)$.

GREEN **delete() Method — Linked List:** The delete operation to remove a node has the time complexity of $O(n)$ in the worst case (**must** traverse the list to find the node). Removing the head node is $O(1)$.

Exam Tip: For singly linked lists, **always** track both 'current' and 'previous' pointers when deleting a middle node.

Exam Trap: Accessing an element by index in a linked list is $O(n)$ — unlike arrays which are $O(1)$. This is a key trade-off.

Chapter 5: Stacks & Queues

Green Highlights

GREEN peek() Method — Stack: Returns the top element from the stack without deleting it from the stack. peek just returns the topmost element; pop() returns and removes it.

GREEN Queue — Time Complexity: The time complexity of enqueue and dequeue operations is $O(1)$. A doubly linked list with FIFO access is the natural implementation of a queue.

GREEN Queue Implementations: Queue can be implemented using: (1) Python list, (2) pointer structures (linked list), (3) doubly linked list, (4) Python collections.deque.

Exam Tip: Stack = LIFO (Last In, First Out). Queue = FIFO (First In, First Out). Know both acronyms.

Exam Tip: Python's collections.deque is the most efficient queue implementation — $O(1)$ for append and popleft.

Exam Trap: pop() on an empty stack returns None (**not** an error) in Python implementation — **always** check for empty stack **before** popping.

Exam Trap: A Python list can simulate a stack but is $O(n)$ for insertions at position 0. Use append/pop for $O(1)$ stack operations.

Chapter 6: Trees & Tree Traversal

Green Highlights

GREEN Binary Tree: A collection of nodes where each node can have zero, one, or two child nodes. A simple binary tree node has a data attribute plus `left_child` and `right_child` pointers.

GREEN Binary Tree Node Creation: To test the binary tree class, four nodes are created: `n1` (root), `n2` (left child), `n3` (right child), `n4` (left grandchild). Linked by assigning `n1.left_child = n2`, etc.

GREEN Pre-order Traversal (DFS): Works as follows: (1) Start at the root node, (2) traverse left sub-tree recursively, (3) traverse right sub-tree recursively. Order: Root → Left → Right.

GREEN Breadth-First Traversal (BFS): Starts from the root of the tree and visits every node on the next level **before** going deeper. Uses a queue internally. Also called level-order traversal.

Exam Tip: Three DFS orders: Pre-order (Root → L → R), In-order (L → Root → R), Post-order (L → R → Root). In-order on a BST gives sorted output.

Exam Trap: BFS uses a queue; DFS uses a stack (or recursion). **Never** confuse which data structure each traversal uses internally.

Chapter 7: Hash Tables & Hashing

Green Highlights

GREEN Hash Table Implementation: To implement a hash table, start by creating a class to hold hash table items. Items **must** have a key and a value. The hash function maps keys to integer indices.

Exam Tip: A perfect hash function maps every key to a unique index with no collisions. In practice, collisions **must** be handled.

Exam Trap: Hash table lookup is $O(1)$ on average but $O(n)$ in the worst case (all keys hash to the same bucket). **Never** assume **always** $O(1)$.

Chapter 8: Graphs & Graph Algorithms

Green Highlights

GREEN Undirected Graph: A graph where the pairings representing edges are unordered. Defined as a collection of vertices $V = \{v_1, v_2, \dots, v_n\}$ with unordered edge pairs $\{v_i, v_j\}$.

GREEN Undirected Graph — Edge Properties: An edge in an undirected graph can be traversed in either direction. An undirected graph is complete if it has the maximum number of edges connecting all vertices.

GREEN Directed Graph — Vertex Degree: The degree of a vertex in a directed graph has two components: out-degree (total edges leaving the vertex) and in-degree (total edges entering the vertex).

GREEN Source and Sink Vertices: Source vertex: a vertex with an in-degree of zero (no incoming edges). Sink vertex: a vertex with an out-degree of zero (no outgoing edges).

GREEN Graph Implementation — Adjacency Lists: Strategy: use a set of graph nodes where each node contains a linked list storing the edges within that node. For weighted graphs, each edge stores the weight.

GREEN Graph Implementation — Adjacency Lists (continued): Uses a set of graph nodes which generate a set of lists. Can be implemented in different ways depending on whether the graph is directed or undirected.

GREEN Adjacency List Representation: Representing the set of vertices adjacent to each vertex as a list, thus generating a set of lists — one list per vertex.

Red Highlights — Graph & Sorting

RED Adjacency List — Vertex List Generation: This involves representing the set of vertices adjacent to each vertex as a list, thus generating a set of lists.

Exam Tip: Adjacency matrix is $O(V^2)$ space; adjacency list is $O(V+E)$ space. Use adjacency list for sparse graphs.

Exam Trap: In-degree and out-degree are **only** defined for directed graphs. An undirected graph **only** has 'degree'. Do **not** mix them up.

Chapter 9: Searching Algorithms

Green Highlights

GREEN Search Algorithm Types: Categorized into two broad types: (1) algorithms applied to a list of items that are already sorted, (2) algorithms applied to an unordered list.

GREEN Linear Search: Simply examines each item in the search pool, one at a time, until either the target is found or the entire pool has been exhausted. Time complexity: $O(n)$.

GREEN Binary Search: A search strategy used to find elements within a sorted array or list. Works by repeatedly halving the search space. Requires the list to be sorted first. Time complexity: $O(\log n)$.

Exam Tip: Binary search requires the list to be sorted. If the list is unsorted, you **must** sort first ($O(n \log n)$) then search $O(\log n)$.

Exam Trap: **Linear** search works on unsorted data; binary search **only** works on sorted data. **Never** apply binary search to an unsorted list.

Chapter 10: Sorting Algorithms

Red Highlights — Chapter Overview

RED **Sorting Algorithms Covered:** The most **important** and popular sorting techniques: Bubble sort, Insertion sort, Selection sort, Quick sort, Heap sort. Comparison of their time complexities is essential.

RED **Sorting — Definition:** Sorting means arranging all items in a list in ascending order of their magnitude. Different algorithms have different performance characteristics.

RED **Heap Sort:** Uses a binary heap data structure. After removing or adding an element, heap order property **must always** be maintained. Time complexity: $O(n \log n)$.

Green Highlights — Algorithm Details

GREEN **Bubble Sort — Mechanism:** Given an unordered list, compare adjacent elements and after each comparison place them in the right order (swap if needed). Largest bubbles to the end each pass.

GREEN **Bubble Sort — Implementation:** Works in a double-nested loop: the inner loop repeatedly compares adjacent elements and swaps them. Time complexity: $O(n^2)$ worst and average case.

GREEN **Insertion Sort — Complexity:** Gives a worst-case runtime complexity of $O(n^2)$ and a best-case complexity of $O(n)$ (when the list is already sorted).

GREEN **Selection Sort:** Begins by finding the smallest element in the list and swapping it with the first position, then repeats for remaining positions. Time complexity: $O(n^2)$ **always**.

GREEN **Quick Sort — Pivot Placement:** After the first iteration of the quick sort algorithm, the chosen pivot point is placed in its correct position in the list. All elements left of pivot are smaller; all right are larger.

GREEN **Sorting Complexity Comparison:** A comparison of sorting algorithm complexities: Bubble/Insertion/Selection = $O(n^2)$ worst. Merge sort = $O(n \log n)$ **always**. Quick sort = $O(n \log n)$ average, $O(n^2)$ worst.

Exam Tip: Merge sort is the most consistent $O(n \log n)$ sorting algorithm. Quick sort is faster in practice but has $O(n^2)$ worst case.

Exam Tip: Selection sort makes exactly $n-1$ swaps regardless of input. Insertion sort is best for nearly-sorted data.

Exam Trap: Quick sort is **NOT always** $O(n \log n)$ — its worst case is $O(n^2)$ when the pivot is **always** the smallest or largest element.

Exam Trap: Bubble sort and Selection sort are **never** efficient for large datasets — both are $O(n^2)$. Do **not** use them in production.

Linked Lists (Chapter 4)

Core Definitions

Linked List: A dynamic data structure consisting of a sequence of nodes, where each node contains data and a pointer/reference to the next node. Size can grow or shrink at runtime.

Node: The basic building block of a linked list. Each node stores data and a reference to the next node. The last node's next pointer is None (null).

Singly Linked List: A linked list where each node has **only** one pointer — to the next node. Traversal is **only** possible in one direction (forward).

Doubly Linked List: A linked list where each node has two pointers: one to the next node and one to the previous node. Enables traversal in both directions.

Head Pointer: A reference to the first node in a linked list. If the head is None, the list is empty. Losing the head pointer means losing access to the entire list.

Exam Tip: Linked lists have $O(1)$ insertion/deletion at the head but $O(n)$ random access. Arrays have $O(1)$ random access but $O(n)$ insertion.

Exam Trap: A linked list is **NOT** an array — it does **not** support index-based access in $O(1)$. Every access requires traversal from the head.

Stacks & Queues — Core Definitions (Chapter 5)

Stack: A LIFO (Last In, First Out) data structure where elements are inserted and removed from the same end (the top). Primary operations: push and pop.

Queue: A FIFO (First In, First Out) data structure where elements are inserted at the rear (enqueue) and removed from the front (dequeue).

Push Operation: Adds a new element to the top of the stack. The top pointer advances to the new node. Time complexity: $O(1)$.

Pop Operation: Removes and returns the topmost element of the stack. Returns None if the stack is empty. Time complexity: $O(1)$.

Enqueue / Dequeue: Enqueue adds an element to the rear of the queue. Dequeue removes an element from the front. Both are $O(1)$ with a doubly linked list.

Exam Tip: The call stack in programming is literally a stack — function calls are pushed; returns pop them off.

Exam Trap: Queue insertion is at the rear; removal is from the front. **Never** insert and remove from the same end (that would be a stack).

Trees — Core Definitions (Chapter 6)

Root Node: The first (topmost) node of a tree from which all other nodes are attached. Every tree has exactly one unique root node. It has no parent.

Leaf Node: A node that does **not** have any children; it is the terminal node of the tree. The degree of a leaf node is **always** 0.

Tree Height: The total number of nodes in the longest path from root to a leaf. Depth of a node = number of edges from the root to that node.

Binary Search Tree (BST): A binary tree where for every node: all values in the left subtree are less than the node, and all values in the right subtree are greater. Enables $O(\log n)$ search.

Exam Tip: In-order traversal of a BST **always** produces elements in sorted (ascending) order.

Exam Trap: A binary tree $\neq$ a binary search tree. A binary tree just limits 2 children; a BST also enforces the ordering property.

Hashing & Graphs — Core Definitions (Ch. 7–8)

Hash Function: A function that maps input data of arbitrary size to a fixed-size integer (the hash value). Used as an index into the hash table array. Hashing converts strings to integers.

Hash Collision: Occurs when two different keys produce the same hash value. Handled using chaining (linked list at each bucket) or open addressing (**linear** probing).

Graph: A non-linear data structure consisting of a set of vertices (nodes) V and a set of edges E connecting pairs of vertices. Models relationships between objects.

Directed Graph (Digraph): A graph where edges have a direction — an edge from v_1 to v_2 does **not** imply an edge from v_2 to v_1 . Each edge is an ordered pair (v_i, v_j) .

Exam Tip: Dictionaries in Python are implemented using hash tables — that is why average lookup is $O(1)$.

Exam Trap: A graph with V vertices can have at most $V(V-1)/2$ edges (undirected). Exceeding this is impossible — do **not** miscalculate.